

RED TEAM RECONNAISSANCE PLAYBOOK

FOR A FINTECH APPLICATION

Individual Contributor Mini-Report — Member 4

Difficulty: Professional | Primary Tool: Metasploit Framework (authorized lab only)

Date: 31 August 2026

Authorization Notice: All testing described in this report was conducted exclusively against an authorized local lab instance (OWASP Juice Shop). No real, live, or production fintech systems were accessed, scanned, or tested.

1. Objective

This document presents the individual contribution of Member 4 to the group's "Red Team Reconnaissance Playbook for a Fintech App" project. The objective was to perform authorized reconnaissance against a fintech-style web application using Metasploit, document findings with severity ratings, and provide clear remediation guidance for each finding — modeled on a realistic, professional red-team engagement.

2. Scope and Authorization

Testing was strictly limited to a self-hosted, local instance of OWASP Juice Shop — an intentionally vulnerable application maintained by OWASP for legal security training. No real, live, third-party, or production systems were scanned or accessed at any point. All activity was performed within an isolated Kali Linux virtual machine under the tester's own control.

3. Lab Environment

Host OS: Windows

Virtualization: Oracle VirtualBox

Attacker Machine: Kali Linux VM (network adapters: Host-Only for host access, NAT for internet access)

Target Application: OWASP Juice Shop (bkimminich/juice-shop), deployed via Docker

Target Address: 127.0.0.1:3000 (localhost, container port-mapped)

Container Runtime: Docker 28.5.2

4. Tool Used

Metasploit Framework v6.4.135-dev was used as the primary reconnaissance tool, specifically the `auxiliary/scanner/http/http_version` module, supplemented with `curl` for direct HTTP header inspection to gather precise evidence.

5. Mini-Plan

- Stand up an authorized local lab target (OWASP Juice Shop via Docker) on an isolated Kali VM.
- Use Metasploit's HTTP scanner modules to perform basic service/version reconnaissance against the target.
- Cross-verify results with direct HTTP header inspection (`curl`) to capture precise, quotable evidence.
- Document any genuine findings with severity ratings, impact, and remediation — no fabricated results.

6. Methodology

- Deployed OWASP Juice Shop locally via Docker to establish a safe, authorized target.
- Loaded `auxiliary/scanner/http/http_version` in Metasploit and configured `RHOSTS/RPORT` against the local target.
- Executed the module to perform banner/version reconnaissance.
- Supplemented the scan with `curl -I` to capture the full raw HTTP response header set, and checked `robots.txt` for additional path disclosure.

- Reviewed headers for information disclosure, misconfiguration, missing security controls, and hardening indicators.
- Documented four genuine findings, each with severity, impact, and remediation.

7. Findings Summary

ID	Finding	Severity
F-01	Overly Permissive CORS Policy (Access-Control-Allow-Origin: *)	Medium
F-02	Information Disclosure via Custom Header (X-Recruiting)	Low
F-03	Missing Content-Security-Policy Header	Low
F-04	Internal Path Disclosure via robots.txt (/ftp)	Low

8. Detailed Findings

F-01: Overly Permissive CORS Policy

Description	The application returns an Access-Control-Allow-Origin: * header on HTTP responses, permitting any origin (any website) to issue cross-origin requests to the API and read the response data through a victim's browser.
Evidence	curl -I http://127.0.0.1:3000 response header: Access-Control-Allow-Origin: *
Impact	In a fintech context, a permissive CORS policy could allow a malicious third-party site to make cross-origin requests on behalf of a logged-in user and read sensitive response data, depending on which endpoints share this same policy. Full exploitability was not confirmed against authenticated endpoints, as this was outside the scope of this recon task.
Severity	Medium
Reason for Severity	Rated Medium rather than High because no proof-of-concept data exfiltration was performed against authenticated endpoints. Rated above Low because CORS misconfiguration is a well-documented, high-impact vulnerability class when combined with sensitive endpoints in a financial application.
Recommended Remediation	Replace the wildcard (*) with an explicit allow-list of trusted origins (e.g. the application's own frontend domain). Never combine a wildcard CORS policy with endpoints that return authenticated or sensitive data.
Evidence Reference	Screenshot: curl -I header output (Kali VM, msfconsole shell), captured during this assessment.

F-02: Information Disclosure via Custom HTTP Header

Description	The server discloses an internal application route through a non-standard response header: X-
-------------	---

	Recruiting: /#/jobs. This reveals a path not otherwise linked or advertised through normal navigation.
Evidence	curl -I http://127.0.0.1:3000 response header: X-Recruiting: /#/jobs
Impact	Low direct impact on its own; no credentials or sensitive data were exposed. However, headers or metadata that reveal internal routes assist an attacker in mapping hidden application functionality during reconnaissance, which can inform follow-on enumeration in a real engagement.
Severity	Low
Reason for Severity	Rated Low rather than Informational because it demonstrates a genuine information-disclosure pattern that directly aided reconnaissance and should not be present in a hardened production deployment.
Recommended Remediation	Remove custom or non-standard headers from production responses. Internal or hidden routes should never be advertised via headers, comments, or other metadata.
Evidence Reference	Screenshot: curl -I header output (Kali VM, msfconsole shell), captured during this assessment.

F-03: Missing Content-Security-Policy Header

Description	The application's HTTP responses do not include a Content-Security-Policy (CSP) header. CSP is a browser-enforced control that restricts which sources scripts, styles, and other resources can load from, mitigating cross-site scripting (XSS) and data-injection attacks.
Evidence	curl -I http://127.0.0.1:3000 full header dump: no Content-Security-Policy entry present among the returned headers.
Impact	Without CSP, the application has no browser-level defense-in-depth against XSS. If any injection vulnerability exists elsewhere in the app, its impact is magnified since the browser has no policy restricting what injected scripts could load or execute.
Severity	Low
Reason for Severity	This is a missing hardening control, not a confirmed exploitable vulnerability — no actual XSS was found or tested. Rated Low (not Informational) because CSP is a widely recommended baseline control, and its absence is a real, actionable gap.
Recommended Remediation	Implement a Content-Security-Policy header restricting script, style, and resource origins to trusted sources only (e.g. default-src 'self'), then test thoroughly since CSP can break functionality if misconfigured.
Evidence Reference	Screenshot: curl -I header output (same as F-01/F-02 evidence) — Content-Security-Policy header absent throughout.

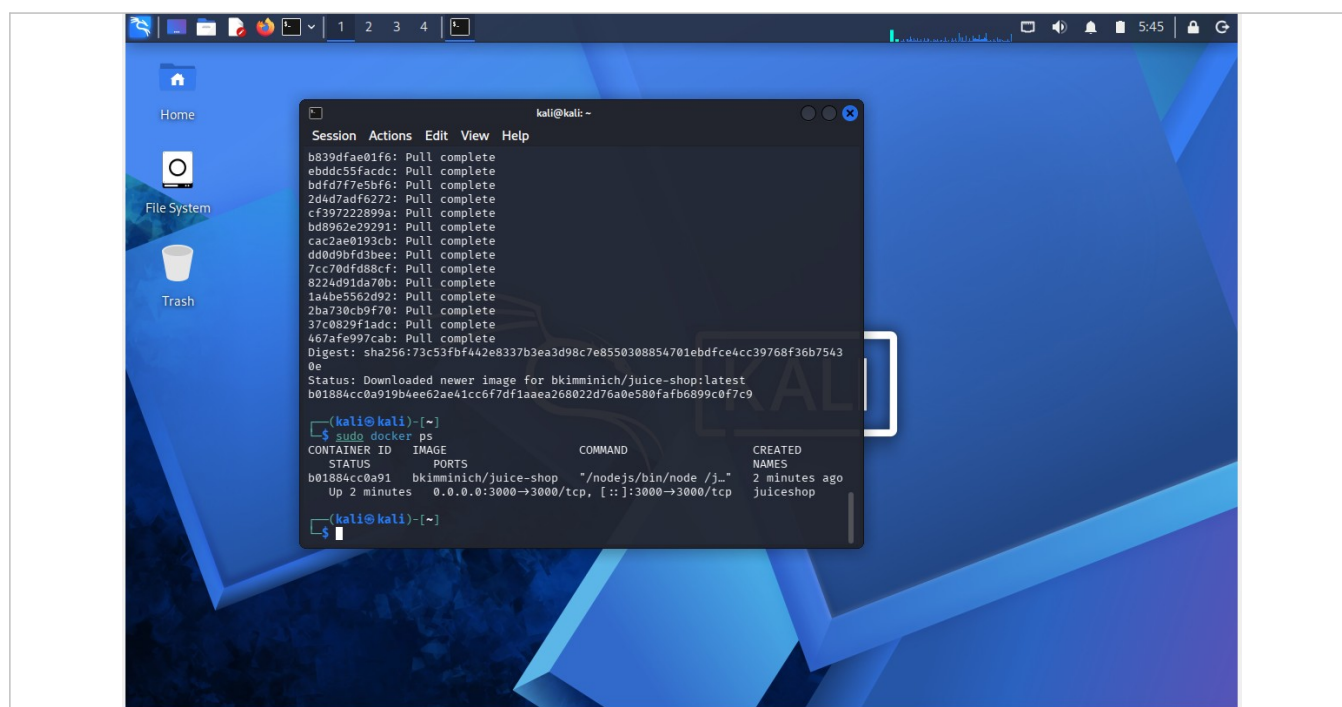
F-04: Internal Path Disclosure via robots.txt

Description	The application's robots.txt file discloses a Disallow: /ftp entry, revealing an internal /ftp path.
--------------------	--

	robots.txt is publicly readable by anyone and is often used by attackers specifically to discover paths developers intended to keep unlisted.
Evidence	curl http://127.0.0.1:3000/robots.txt response: User-agent: * Disallow: /ftp
Impact	Reveals a hidden application path not otherwise linked anywhere. Combined with F-02 (the X-Recruiting header), this shows a pattern of internal paths leaking through metadata rather than being properly access-controlled — useful reconnaissance data for an attacker mapping the app's attack surface.
Severity	Low
Reason for Severity	No credentials or sensitive data directly exposed — impact is limited to aiding further reconnaissance, consistent with F-02's rating.
Recommended Remediation	Do not rely on robots.txt to hide sensitive or internal paths — it is not an access control mechanism. Any path that shouldn't be publicly accessible should be protected by proper authentication/authorization, not obscurity.
Evidence Reference	Screenshot: curl http://127.0.0.1:3000/robots.txt output (Kali VM, msfconsole shell), captured during this assessment.

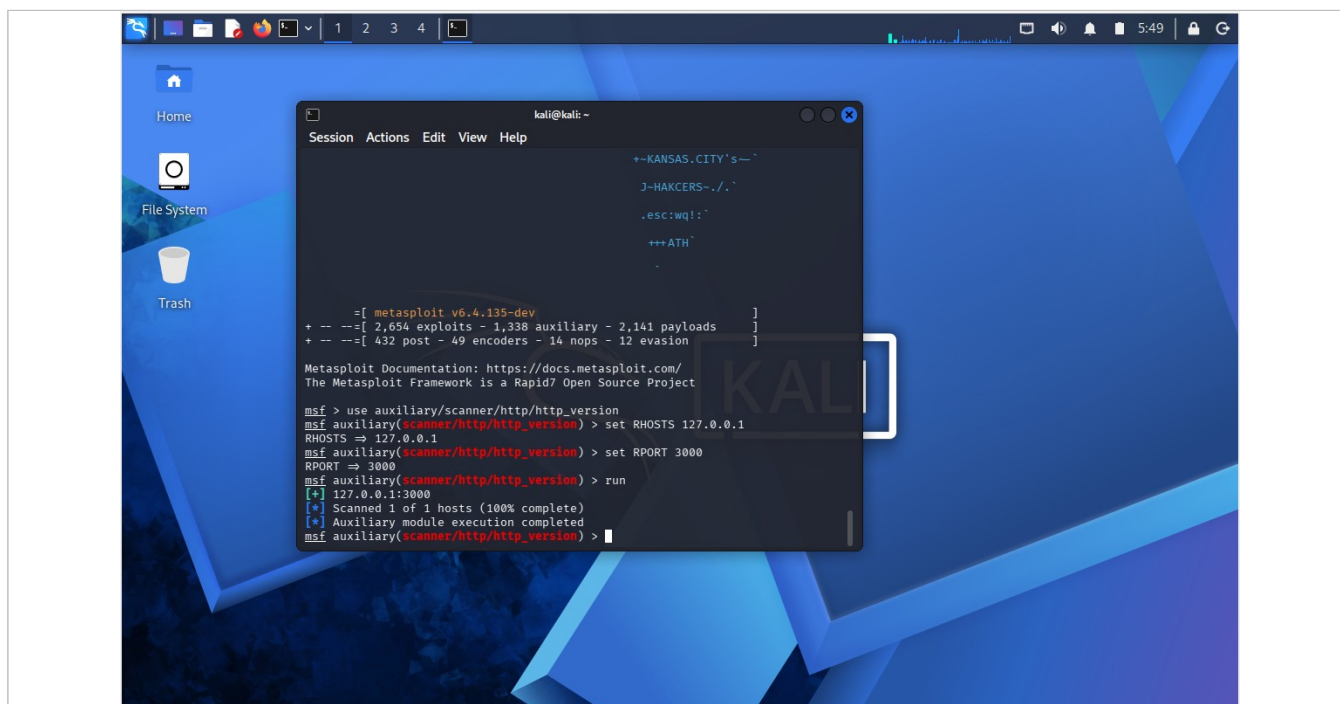
9. Evidence and Screenshots

Screenshot 1 — Lab environment: Juice Shop container running and port-mapped, confirming an authorized, isolated target.



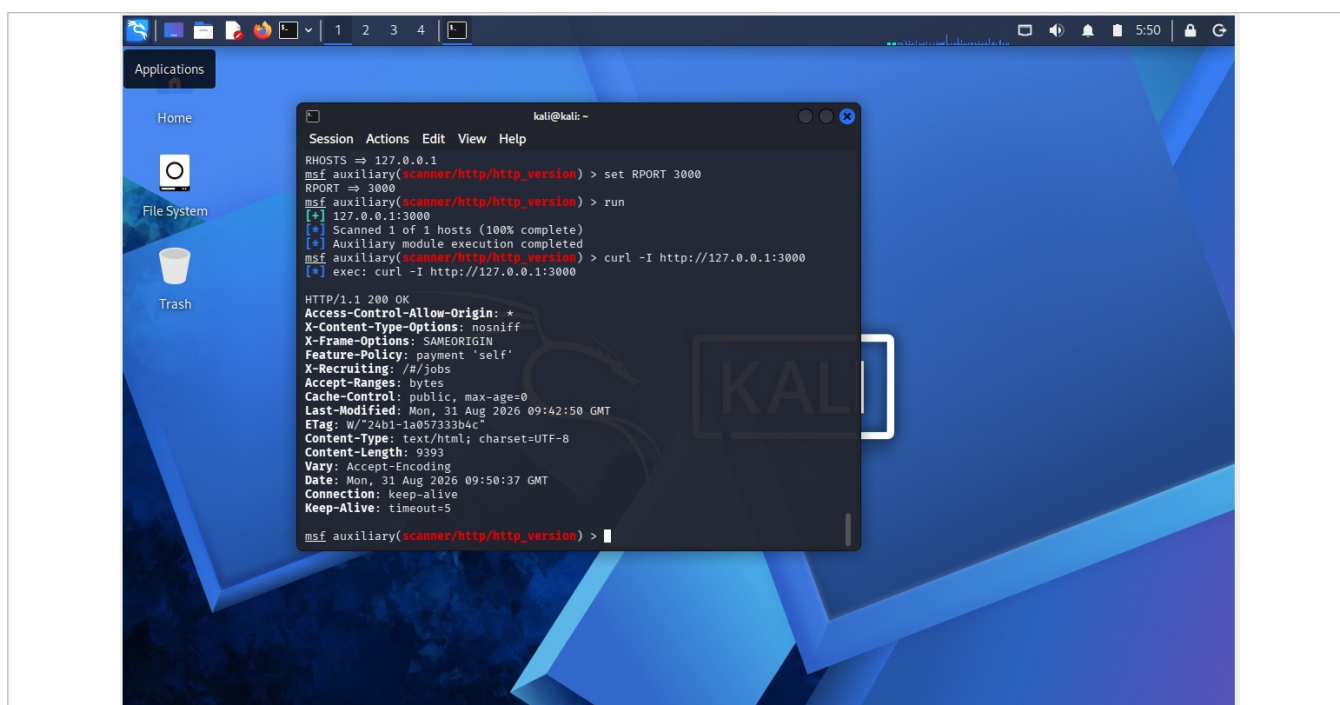
Screenshot 1: docker ps output — Juice Shop container up, 0.0.0.0:3000->3000/tcp

Screenshot 2 — Metasploit auxiliary/scanner/http/http_version module executed against the authorized lab target.



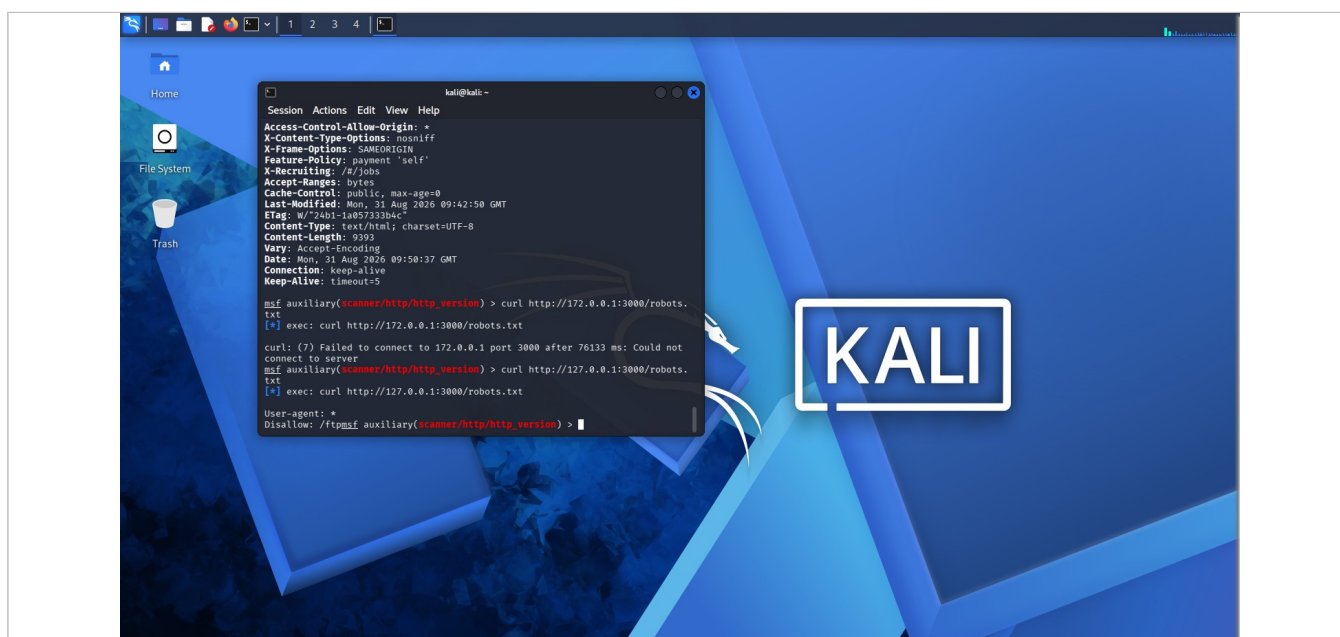
Screenshot 2: Metasploit scan run against 127.0.0.1:3000

Screenshot 3 — Raw HTTP response headers, the primary evidence source for F-01, F-02, and F-03.



Screenshot 3: curl -I http://127.0.0.1:3000 header output

Screenshot 4 — robots.txt output, disclosing the internal /ftp path (evidence for F-04).



Screenshot 4: curl http://127.0.0.1:3000/robots.txt output

10. Overall Impact

Individually, none of these findings represent a confirmed breach. Together, they indicate consistent gaps in HTTP-layer hardening — a permissive CORS policy, information leaking through headers and metadata (X-Recruiting, robots.txt), and a missing browser-level XSS defense (CSP). In a real fintech deployment, this pattern could widen the attack surface for cross-origin data exposure and reconnaissance-driven follow-on attacks. All four are low-effort to remediate and should be addressed before any production exposure.

11. Remediation Recommendations

- Restrict CORS to an explicit allow-list of trusted origins; never use a wildcard alongside sensitive or authenticated endpoints.
- Strip non-standard, informational headers (e.g. X-Recruiting) from production responses.
- Implement a Content-Security-Policy header to provide browser-level defense-in-depth against XSS.
- Do not use robots.txt to hide internal or sensitive paths; enforce proper authentication/authorization instead.
- Adopt a standard security-header baseline (e.g. OWASP Secure Headers Project) and re-verify with curl or a header-scanning tool after remediation.

12. Conclusion

This assessment successfully demonstrated a basic, authorized HTTP reconnaissance workflow against a fintech-style web application using Metasploit, producing four genuine, evidence-backed findings with clear severity ratings and remediation guidance. The exercise reflects a realistic first-pass recon step in a professional red-team engagement, appropriately scoped to a safe lab environment.

13. References

OWASP Juice Shop Project — <https://owasp.org/www-project-juice-shop/>

Metasploit Framework Documentation — <https://docs.metasploit.com/>

OWASP Secure Headers Project — <https://owasp.org/www-project-secure-headers/>

MDN: Cross-Origin Resource Sharing (CORS) — <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>